Lists and Tuples

Eunji Kim
eunjikim@cau.ac.kr

List

List is used to store multiple items in a single variable.

```
mylist = ["apple", "banana", "cherry"]
```

> 📝 Lists are one of 4 built-in data types in Python used to store collections of data, the other 3 are Tuple, Set, and Dictionary, all with different qualities and usage.

Here, we will cover how to create, access, and update lists.

Create a List

To create a list, enclose a set of items in square brackets `[]`:

```
thislist = ["apple", "banana", "cherry"]
print(thislist)
```

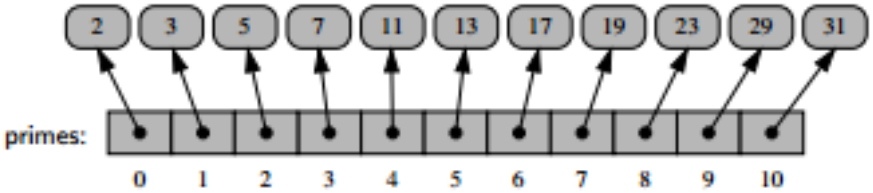
> 📝 To check the length of a list, use `len()`:
>
> ```
> print(len(thislist)) # 3
> ```

Access a List

You can access single or multiple items in a list using **indexing** and **slicing**.

Indexing

You can access single item in list using index.




```
primes_list = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31]
```

- Index starts with `0`.
- Items can be accessed by indexing with brackets `[]`.

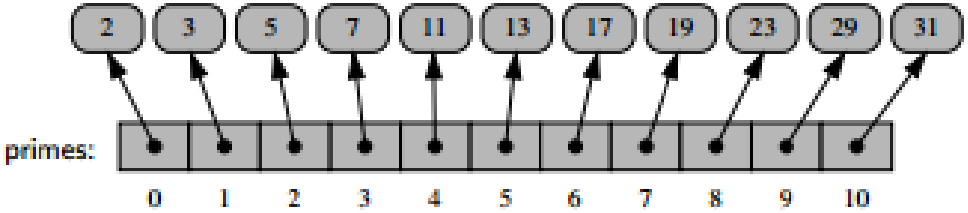
```
print(primes_list[3]) # 7
```

- **Negative Indexing** starts from the end
  - `-1` refers to the last item, `-2` refers to the second last item etc.

```
print(primes_list[-3]) # 23
```

Slicing

You can access a range of items by using the slice syntax.




```
# We pass slice instead of index like this: `[start:end]`.
# The search will start at index `7` (included) and end at index `9` (not included).
print(primes_list[7:9])  # [19, 23]
```

```
# Slice From the Start or To the End
print(primes_list[:5])   # [2, 3, 5, 7, 11]
print(primes_list[5:])   # [13, 17, 19, 23, 29, 31]

# Forward and backward indexing can be used together.
print(primes_list[1:-3]) # [3, 5, 7, 11, 13, 17, 19]

# We can also define the step, like this: `[start:end:step]`.
print(primes_list[0:10:2]) # [2, 5, 11, 17, 23]
```

Update Lists

List is **mutable**; you can change, add, or remove items.

Change Single Item Value

To change the value of a specific item, refer to the index number:

```
thislist = ["apple", "banana", "cherry"]
thislist[1] = "blackcurrant"
print(thislist) # ["apple", "blackcurrant", "cherry"]
```

Change a Range of Item Values

To change the value of items within a specific range, use the *slicing* syntax:

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "mango"]
thislist[1:3] = ["blackcurrant", "watermelon"]
print(thislist) # ["apple", "blackcurrant", "watermelon", "orange", "kiwi", "mango"]
```

If you insert **more** items than you replace, the new items will be inserted where you specified, and the remaining items will move accordingly:

```
thislist = ["apple", "banana", "cherry"]
thislist[1:2] = ["blackcurrant", "watermelon"]
print(thislist) # ["apple", "blackcurrant", "watermelon", "cherry"]
```

If you insert **less** items than you replace, the new items will be inserted where you specified, and the remaining items will move accordingly:

```
thislist = ["apple", "banana", "cherry"]
thislist[1:3] = ["watermelon"]
print(thislist) # ["apple", "watermelon"]
```

Add Single Item to List

Using the `append()` method to append an item:

```
thislist = ["apple", "banana", "cherry"]
thislist.append("orange")
print(thislist) # ['apple', 'banana', 'cherry', 'orange']
```

To insert a list item at a specified index, use the `insert()` method.

```
thislist = ["apple", "banana", "cherry"]
thislist.insert(1, "orange")
print(thislist) # ['apple', 'orange', 'banana', 'cherry']
```

Add Multiple Items to List

To append multiple elements to the current list, use the `extend()` method.

```
thislist = ["apple", "banana", "cherry"]
thislist.extend(["mango", "pineapple", "papaya"])
print(thislist) # ["apple", "banana", "cherry", "mango", "pineapple", "papaya"]
```

You can also use `+` operator.

```
thislist = ["apple", "banana", "cherry"]
thislist += ["mango", "pineapple", "papaya"]
# or
# thislist = this_list + ["mango", "pineapple", "papaya"]
print(thislist) # ["apple", "banana", "cherry", "mango", "pineapple", "papaya"]
```

Remove Specified Item in List

The `remove()` method removes the specified item.

```
thislist = ["apple", "banana", "cherry"]
thislist.remove("banana")
print(thislist) # ["apple", "cherry"]
```

> ⚠️ If there are more than one item with the specified value, the `remove()` method removes the first occurrence:
>
> ```
> thislist = ["apple", "banana", "cherry", "banana", "kiwi"]
> thislist.remove("banana")
> print(thislist) # ["apple", "cherry", "banana", "kiwi"]
> ```

Remove Specified Index in List

The `pop()` method removes the specified index.

```
thislist = ["apple", "banana", "cherry"]
thislist.pop(1)
print(thislist) # ["apple", "cherry"]
```

If you do not specify the index, the `pop()` method removes the last item.

```
thislist = ["apple", "banana", "cherry"]
thislist.pop()
print(thislist) # ["apple", "banana"]
```

Del keyword

The `del` keyword also removes the specified index:

```
thislist = ["apple", "banana", "cherry"]
del thislist[0]
print(thislist) # ["banana", "cherry"]
```

> 📝 The `del` keyword can also delete the list completely.
>
> ```
> thislist = ["apple", "banana", "cherry"]
> del thislist
> ```

Sort Lists

List objects have a `sort()` method that will sort the list alphanumerically, ascending, by default:

Sort the list alphabetically:

```
thislist = ["orange", "mango", "kiwi", "pineapple", "banana"]
thislist.sort()
print(thislist) # ['banana', 'kiwi', 'mango', 'orange', 'pineapple']
```

Sort the list numerically:

```
thislist = [100, 50, 65, 82, 23]
thislist.sort()
print(thislist) # [23, 50, 65, 82, 100]
```

To sort descending, use the keyword argument `reverse=True`:

```
thislist = ["orange", "mango", "kiwi", "pineapple", "banana"]
thislist.sort(reverse=True)
print(thislist) # ['pineapple', 'orange', 'mango', 'kiwi', 'banana']
```

Sort the list numerically:

```
thislist = [100, 50, 65, 82, 23]
thislist.sort(reverse=True)
print(thislist) # [100, 82, 65, 50, 23]
```

Tuple

Tuple

Tuple is used to store multiple items in a single variable.

To create a tuple, enclose a set of items in parenthesis `()`.

```
thistuple = ("apple", "banana", "cherry")
```

> **Difference between List and Tuple**
>
> - List is Mutable and Tuple is not
>   (Cannot update an element, insert and delete elements)
> - Tuples are faster because of read only nature.
>   Memory can be efficiently allocated and used for tuples.

Tuples are Immutable!

Once a tuple is created, you cannot change its values; you *cannot change, add, or remove* items.

```
x = ["apple", "banana", "cherry"] # list
x[1] = 'kiwi' # now x becomes ["apple", "banana", "cherry"]

x = ("apple", "banana", "cherry") # tuple
x[1] = 'kiwi' # TypeError: 'tuple' object does not support item assignment
```

Change Items in Tuple

Tuples are immutable (unchangable). But there is a workaround. You can convert the tuple into a list, change the list, and convert the list back into a tuple.

For example, we can change an item value in a tuple as below:

```
x = ("apple", "banana", "cherry")
y = list(x) # casting to list
y[1] = "kiwi"
x = tuple(y) # casting to tuple
print(x) # ("apple", "kiwi", "cherry")
```

Append Items in Tuple

Since tuples are immutable, they do not have a built-in `append()` method.

But there are workarounds to add or remove items in a tuple.

1. **Convert into a list**: Just like the workaround for changing a tuple, you can convert it into a list, add your item(s), and convert it back into a tuple.

```
thistuple = ("apple", "banana", "cherry")
y = list(thistuple)
y.append("orange")
thistuple = tuple(y) # ("apple", "banana", "cherry", "orange)
```

2. **Add tuple to a tuple**. You are allowed to add tuples to tuples, so if you want to add one item, (or many), create a new tuple with the item(s), and add it to the existing tuple:

```
thistuple = ("apple", "banana", "cherry")
y = ("orange",)
thistuple += y
print(thistuple) # ("apple", "banana", "cherry", "orange")
```

> 📝 When creating a tuple with only one item, remember to include a comma after the item, otherwise it will not be identified as a tuple.
>
> ```
> x = (3)  # int (identical to `x = 3`)
> x = (3,) # tuple
> ```

Remove Items in Tuple

Since tuples are immutable, they do not have a built-in `remove()` method, but you can use the same workaround as we used for changing and adding tuple items:

```
thistuple = ("apple", "banana", "cherry")
y = list(thistuple)
y.remove("apple")
thistuple = tuple(y) # ("banana", "cherry")
```

List vs. Tuple

- List: Mutable and Iterable
  - You can iterate over a list and also modify its elements.

```
    my_list = [1, 2, 3] # Mutable and Iterable
    for item in my_list:
        print(item)
    my_list[0] = 10 # Modifying the list
```

- Tuple: Immutable and Iterable
  - You can iterate over them, but you cannot change their elements after creation.

```
my_tuple = (1, 2, 3) # Immutable and Iterable
for item in my_tuple:
    print(item)
# my_tuple[0] = 10 # This would raise a TypeError
```

Thank You.